SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME:Software Testing **COURSE CODE: CSA3711**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

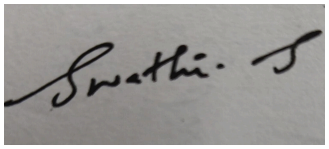
9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5

14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5
15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

I Swathi S certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Answers:

1.Online Banking System – Fund Transfer Failures

An online banking system allows users to transfer money from one account to another. Frequent failures during fund transfers, especially near the minimum and maximum transfer limits, indicate that the existing testing strategy is insufficient. The testing team may be focusing mainly on normal transaction amounts without properly checking boundary values. Therefore, black-box testing techniques such as Equivalence Class Partitioning (ECP) and Boundary Value Analysis (BVA) should be applied to improve test coverage and identify defects.

Equivalence Class Partitioning divides input values into groups that are expected to behave similarly. For example, if the permitted transfer amount is between ₹100 and ₹50,000, the valid input class includes amounts from ₹100 to ₹50,000. Invalid input classes include amounts below ₹100, amounts above ₹50,000, zero, and negative values. Instead of testing every possible amount, the tester selects representative values from each class. This reduces the number of test cases while maintaining useful input coverage.

Boundary Value Analysis focuses on testing values at the limits and immediately around them because errors frequently occur at boundaries. The tester should check ₹99, ₹100, ₹101, ₹49,999, ₹50,000, and ₹50,001. The expected result is that amounts within the permitted range are accepted, provided all other banking conditions are satisfied, while amounts outside the range are rejected. The tester should also verify insufficient account balance, invalid beneficiary details, and unauthorized transactions.

Random testing involves selecting input values randomly. Although it can discover unexpected defects, it may fail to select important boundary values and does not guarantee systematic coverage. Therefore, random testing should be retained as a supplementary technique rather than used as the primary testing method. The improved strategy should combine ECP, BVA, input validation, and regression testing. The tester should also verify that account balances are updated correctly and that failed transactions do not cause incorrect deductions.

In conclusion, combining ECP and BVA provides a structured method for identifying input-related defects in online banking systems. These techniques improve test coverage, reduce redundant test cases, and help detect failures near transfer limits. Retaining random testing as an additional technique further improves defect detection and increases the reliability and security of fund transfers.

Important Test Cases:

- Test the minimum permitted transfer amount.
- Enter an amount below the minimum limit.
- Test the maximum permitted transfer amount.
- Enter an amount above the maximum limit.
- Attempt to transfer more money than the available balance.

2.Healthcare Application – Critical Defects After Deployment

A healthcare application processes important information such as patient details, medical records, prescriptions, and laboratory results. The discovery of critical defects after deployment indicates that the testing team may not have adequately tested all important execution paths. Insufficient test coverage can allow errors to remain hidden even when the application appears to work correctly during normal operations. Therefore, white-box testing techniques should be used to examine the internal program logic and identify untested statements, branches, and paths.

White-box testing examines the internal structure and control flow of a program. One important technique is the Control Flow Graph (CFG), which represents the execution flow using nodes and directed edges. Consider a patient registration module that reads patient details and checks whether all required information is valid. The program begins by accepting the patient's details and evaluating the validation condition. If the information is valid, the system saves the patient record. If the information is invalid, the system displays an error message. The CFG contains separate branches for these two outcomes, and both branches must be tested to verify the correctness of the module.

Statement coverage measures whether every executable statement has been executed at least once. However, achieving 100% statement coverage does not guarantee that every decision outcome has been tested. For example, a test case containing valid patient details may execute the statements required to save a record but fail to test the error-handling branch. Branch coverage addresses this limitation by checking whether every possible decision outcome, such as True and False, has been executed. Path coverage examines different execution paths through the program and helps identify defects that occur only under particular sequences of decisions. Complete path coverage may be impractical in programs containing loops, so important and feasible paths should be prioritized.

To improve the testing strategy, the team should first construct a CFG for the healthcare module and identify all decision points. Test cases should then be designed for valid patient records, missing information, invalid input values, and other relevant conditions. The team should measure statement and branch coverage to identify untested sections of the code. Important combinations of conditions and error-handling paths should also be tested. Since healthcare applications handle sensitive information, the team should additionally verify data integrity, access control, and correct processing of patient records. Regression testing should be performed after defects are corrected to ensure that existing functionality continues to work properly.

In conclusion, the healthcare application requires a structured white-box testing approach that combines statement coverage, branch coverage, and selected path testing. Statement coverage alone is not sufficient to ensure the correctness of the application. By identifying missing branches and important execution paths, the testing team can detect hidden defects, improve reliability, protect patient information, and reduce the risk of failures after deployment.

Important Test Cases:

- Enter valid patient details.
- Leave mandatory fields empty.
- Enter an invalid patient ID.

- Test both true and false decision branches.
- Test different execution paths in the application.

3.Retail E-Commerce System – Errors in Input Combinations

A retail e-commerce system calculates the final price of a product using its price, quantity, and discount. Errors may occur when these inputs interact. For example, a discount might be applied twice, an invalid quantity might be accepted, or an incorrect total might be calculated. The failure to detect these defects indicates that the existing black-box testing strategy may be testing individual inputs without sufficiently testing combinations of inputs.

Equivalence Class Partitioning (ECP) divides inputs into valid and invalid groups. For example, positive product prices may form a valid class, while zero or negative prices may form invalid classes if the system requires a positive price. Similarly, positive whole-number quantities are valid when fractional quantities are not allowed, while zero, negative quantities, and fractions are invalid. Discount values must also remain within the range permitted by the system. Testing representative values from each class helps identify input validation errors without testing every possible value.

Boundary Value Analysis (BVA) should be used to test values near important limits. If the maximum permitted discount is 50%, the tester should check 49%, 50%, and 51%. The same approach should be applied to minimum prices, maximum quantities, and any other defined limits. Decision Table Testing is also useful because the final price depends on combinations of price, quantity, and discount. A decision table can describe normal purchases, bulk purchases with discounts, zero quantities, invalid prices, and discounts beyond the permitted limit. Each combination should have a clearly defined expected result.

The improved testing strategy should combine ECP, BVA, and Decision Table Testing. The tester should verify that the total is calculated correctly, that discounts are applied only when their conditions are satisfied, and that invalid inputs are rejected. Pairwise testing may also be used to reduce the number of combinations when many input fields exist, but important business rules and critical combinations must still be tested explicitly. Regression testing should be performed after fixing defects.

In conclusion, testing price, quantity, and discount separately is insufficient because defects may occur only when these values interact. Combining ECP, BVA, and Decision Table Testing improves input coverage and helps identify calculation and validation errors. This structured approach increases test completeness while reducing unnecessary repetition.

Important Test Cases:

- Purchase an item at its regular price.
- Enter the minimum permitted quantity.
- Enter a quantity above the permitted limit.
- Apply a valid discount to an eligible purchase.
- Enter an invalid discount code.

4.Software Module with Complex Nested Conditions and Loops

A software module containing nested conditions and loops may have many possible execution paths. If the testing team reports only partial path coverage, some branches, loop conditions, or combinations of decisions may remain untested. This can allow defects to appear when the program receives particular inputs or executes a specific sequence of operations. Therefore, the current white-box testing approach must be improved through systematic Control Flow Graph construction and path testing.

A Control Flow Graph (CFG) represents the sequence of operations in a program. Each node represents a statement or decision, and each directed edge represents a possible transition between operations. In a module containing nested conditions, the CFG should include every True and False decision outcome. For loops, it should represent the entry condition, loop body, repeated execution, and exit condition. If the CFG is incomplete, some important execution paths may not be identified, resulting in insufficient testing.

Path testing involves designing test cases to execute different paths through the program. The team should identify independent paths, which are paths that introduce at least one new edge not included in the previously selected paths. Cyclomatic complexity can help estimate the number of linearly independent paths. It can be calculated using the formula , where is the number of edges, is the number of nodes, and is the number of connected components, usually one for a single connected program. For a connected CFG, the complexity can also be calculated as the number of decision nodes plus one.

The improved strategy should begin by constructing and checking the CFG. The team should then calculate cyclomatic complexity, identify the independent paths, and create test cases for each feasible path. Loop testing should include zero iterations, one iteration, multiple iterations, and boundary conditions around the maximum permitted number of iterations. Nested conditions should be tested with inputs that exercise both outcomes of each decision. Infinite loops, incorrect loop termination, and unexpected exits should receive special attention.

In conclusion, partial path coverage may leave important logical defects undetected. A systematic CFG-based strategy, independent path testing, and loop boundary testing provide stronger coverage of complex modules. Cyclomatic complexity helps estimate the testing effort, while testing critical feasible paths improves reliability and reduces the possibility of failures in production.

5.Financial Application – Combining Black-Box and White-Box Testing

A financial application may use both black-box and white-box testing but still contain defects that appear after deployment. This indicates that the testing techniques may not be properly integrated or that the test cases and coverage measurements are insufficient. Financial applications perform important operations such as calculating interest, processing payments, transferring funds, and updating account balances. Even a small calculation or logic error can cause incorrect financial results. Therefore, a more systematic hybrid testing strategy is required.

Black-box testing verifies the application's behaviour without examining its internal code. Equivalence Class Partitioning (ECP) divides input values into valid and invalid groups, while Boundary Value Analysis (BVA)

checks values at and around important limits. For example, in a payment module, the tester should verify valid payment amounts, zero amounts, negative amounts, and amounts immediately below or above the permitted transaction limit. These techniques help identify input validation errors and incorrect handling of boundary values.

White-box testing examines the internal program logic. Control Flow Graphs (CFGs) represent statements, decisions, and execution paths. Path testing helps identify untested logical routes, while statement and branch coverage measurements show which parts of the program have been executed. High statement coverage alone does not prove that every decision outcome has been tested. Therefore, the team must examine both coverage measurements and the actual quality of the test cases.

The improved hybrid strategy should begin by identifying important financial requirements and designing black-box test cases for valid inputs, invalid inputs, and boundary conditions. The team should then construct CFGs for critical functions and test important independent paths and decision outcomes. Test cases should also verify account balance updates, transaction failure handling, duplicate transactions, and recovery from errors. Integration testing should check communication between modules, and regression testing should confirm that fixes do not introduce new defects. Coverage results should be reviewed alongside the number and severity of defects found.

In conclusion, combining black-box and white-box testing provides a more complete testing approach than relying on either technique alone. Black-box testing checks whether the application meets user requirements, while white-box testing examines its internal logic. Using ECP, BVA, CFGs, path testing, and suitable coverage criteria improves defect detection and reduces the risk of financial errors in production.

Important test cases:

- Enter valid financial transaction details.
- Enter an invalid transaction amount.
- Test values at the minimum and maximum boundaries.
- Execute different branches in the transaction logic.
- Test invalid transactions through the interface and internal logic.

6.HR Payroll System – Incorrect Bonus Payouts

An HR payroll system calculates monthly bonuses based on employees' years of service and performance ratings. The testing team discovered that employees with exactly 5, 10, and 15 years of service were receiving incorrect bonus amounts. This indicates that the existing testing approach did not properly check boundary values. Since payroll calculations affect employee payments, the system must be tested carefully to ensure that bonuses are calculated accurately according to the company's rules.

Equivalence Class Partitioning (ECP) divides input values into valid and invalid groups. In this system, employees can be grouped according to their years of service and performance ratings. For example, the service-year groups may be below 5 years, from 5 to below 10 years, from 10 to below 15 years, and 15 years or more, depending on the actual bonus policy. Invalid inputs, such as negative service years or unsupported performance ratings, should also be tested. This helps verify whether the system correctly applies the bonus rules to each group.

Boundary Value Analysis (BVA) focuses on values immediately below, at, and above important limits. The tester should check 4, 5, and 6 years; 9, 10, and 11 years; and 14, 15, and 16 years. The expected bonus for each value should be determined using the official payroll requirements. Exploratory testing can discover unexpected defects, but it may not consistently cover every milestone. Therefore, ECP and BVA should be used as the main techniques, with exploratory testing as a supporting method.

The improved strategy should include systematic boundary testing, performance-rating combinations, validation of calculated bonus amounts, and regression testing after defects are fixed. The tester should compare actual payouts with expected payouts to confirm correctness.

In conclusion, ECP and BVA provide a structured and repeatable approach to testing payroll calculations. They help identify errors at important service-year boundaries, prevent incorrect payouts, and improve the reliability of the payroll system.

7.Laboratory Temperature Control System – Alarm Failure

An automated temperature control system manages laboratory temperature using heaters, coolers, and alarms. After deployment, the system failed to activate the alarm during a sensor-malfunction sequence. This indicates that the testing team may not have tested all important decision outcomes and combinations of conditions. Since incorrect temperature control can affect laboratory equipment and samples, the system requires thorough white-box testing.

A Control Flow Graph (CFG) represents the execution flow of a program using nodes and directed edges. In this system, the program first checks whether the temperature sensor is functioning correctly. If the sensor is faulty, the system should activate the required alarm or enter a safe state. If the sensor is working, the program checks whether the temperature is too low, within the normal range, or too high. Depending on the result, it activates the heater, maintains normal operation, or activates the cooler. The CFG should represent these decisions and their possible outcomes.

Branch coverage checks whether every decision outcome has been executed. However, Modified Condition/Decision Coverage (MC/DC) is more suitable for complex Boolean decisions because it verifies that each individual condition can independently affect the overall decision outcome. For example, if an alarm decision depends on sensor failure, unsafe temperature, and an emergency condition, the tester should design test cases that demonstrate the independent effect of each condition according to the actual program logic.

The improved testing strategy should include tests for normal temperature, high temperature, low temperature, sensor failure, and combinations of these conditions. The team should verify that alarms activate when required, that the system enters a safe state during sensor failure, and that faults are recorded correctly. Coverage results should be reviewed, and regression testing should be performed after corrections. For additional confidence, the system can also be tested with simulated sensor faults and actual hardware.

In conclusion, CFG construction, branch coverage, and MC/DC testing help identify missing logical conditions and untested execution paths. These techniques improve the reliability of the temperature control system and reduce the risk of safety-related failures.

Important test cases:

- Keep the temperature within the permitted range.
- Set the temperature at the upper threshold.
- Increase the temperature above the upper threshold.
- Decrease the temperature below the lower threshold.
- Test individual conditions as both true and false for MC/DC coverage.

8.Travel Booking Portal – Negative Total Amount

A travel booking portal allows users to apply promotional codes based on booking price, promotional type, and user subscription tier. The system sometimes produces a negative total amount when certain promotional codes are combined. This suggests that the application may be applying discounts incorrectly or failing to validate combinations of promotional offers. Testing each input separately is insufficient because the defect may occur only when multiple conditions interact.

Decision Table Testing is a black-box technique used to test different combinations of conditions and their expected actions. In this system, the conditions may include whether the booking price is valid, whether a promotional code is valid, whether multiple codes can be combined, and whether the user is eligible for a particular discount. The expected actions include applying the permitted discount, rejecting an invalid combination, and calculating the final payable amount. For example, a valid booking with an eligible promotional code should receive the allowed discount, while an expired code should be rejected.

Cause-Effect Graphing can also be used to identify relationships between input conditions and system outputs. The tester should examine valid and invalid booking prices, different promotional types, user tiers, and combinations of multiple codes. Boundary Value Analysis should be used to test minimum booking amounts and maximum discount limits. The system must ensure that the final payable amount never becomes negative and that the displayed total matches the amount sent to the payment gateway.

The improved testing strategy should prioritize important business-rule combinations while avoiding unnecessary duplicate test cases. Pairwise testing can help reduce the number of combinations when many input fields exist, but critical promotional rules must still be tested explicitly. Regression testing should be conducted after fixing defects to ensure that existing promotional offers continue to work correctly.

In conclusion, Decision Table Testing and Cause-Effect Graphing provide a systematic way to test interactions between booking prices, promotional codes, and user tiers. Combining these methods with boundary testing improves defect detection, reduces redundant test cases, and prevents incorrect booking totals.

9.Legacy Database Migration Tool – Infinite Loop Crashes

A legacy database migration tool validates and transfers records between databases. Although the testing team reports high statement coverage, the application still experiences infinite-loop crashes in production. This indicates that executing every statement at least once was not sufficient to verify loop conditions and termination behaviour. The program may execute all statements successfully for ordinary records but fail when it encounters invalid data or particular combinations of inputs.

Statement coverage measures whether every executable statement has been executed at least once. However, it does not guarantee that loops terminate correctly. For example, a loop may process records correctly but fail to update its counter when an invalid record is encountered. As a result, the same record may be processed repeatedly, causing an infinite loop. Therefore, the testing team must examine loop conditions, counter updates, exit conditions, and error-handling logic.

Path Testing should be used to identify independent execution paths through the program. A Control Flow Graph (CFG) should represent the validation loops, decisions, and exit conditions. Cyclomatic complexity can be calculated using the formula , where is the number of edges, is the number of nodes, and is the number of connected components. For a single connected graph, it can also be calculated as the number of decision nodes plus one. This value helps estimate the number of linearly independent paths to consider during basis path testing.

The improved testing strategy should include zero records, one record, multiple records, maximum permitted records, invalid records, and empty datasets. The tester should verify that each loop makes progress or terminates correctly and that invalid records do not cause repeated processing. Nested loops should be tested with different iteration counts. Large datasets and timeout checks can help identify performance problems and non-terminating behaviour. Regression testing should be performed after corrections.

In conclusion, high statement coverage alone cannot guarantee correct loop behaviour. CFG-based path testing, cyclomatic complexity analysis, and loop boundary testing help identify infinite-loop defects. These techniques improve migration reliability and reduce the risk of incomplete data transfers or processing failures.

Important test cases:

- Migrate a database containing no records.
- Migrate a database containing one record.
- Migrate a database containing multiple records.
- Test loops at their minimum and maximum expected iterations.
- Test different execution paths to identify infinite loops.

10.Mobile Food Delivery App – Tracker Stuck During Driver Cancellation

A mobile food delivery application tracks orders through different states, such as Ordered, Preparing, Out for Delivery, and Delivered. Users reported that the tracker becomes stuck when a driver cancels during delivery. This indicates that the existing black-box testing strategy may not adequately cover state changes

and exceptional events. Testing individual input values cannot verify every sequence of events in a dynamic workflow. Therefore, State Transition Testing is an appropriate technique for this problem.

State Transition Testing examines how an application moves from one state to another when an event occurs. In the delivery application, an order normally moves from Ordered to Preparing, then to Out for Delivery, and finally to Delivered. If the driver cancels during delivery, the application should follow a defined cancellation or reassignment process. It should not remain permanently in the Out for Delivery state. The expected behaviour should follow the application's business rules, including whether another driver is assigned, the customer is notified, or a refund is initiated.

The testing team should prepare a state transition diagram or table showing the possible states, triggering events, valid transitions, and expected results. Test cases should cover normal order progression, driver cancellation, replacement driver assignment, permitted customer cancellation, delivery completion, and network interruptions during a state change. Invalid transitions should also be tested. For example, the system should not mark an order as Delivered before the required delivery process is completed.

Equivalence Class Partitioning and Boundary Value Analysis are useful for testing input values, but they may not detect defects caused by event sequences or invalid state transitions. State Transition Testing focuses directly on these behaviours. The team should also test repeated cancellation events, delayed notifications, and concurrent updates from the customer and driver applications. Regression testing should verify that the normal tracking process continues to work after the defect is corrected.

In conclusion, State Transition Testing helps identify defects in workflows that depend on changing states and events. By testing normal transitions, exceptional events, and invalid state changes, the team can prevent the tracker from becoming stuck and improve the reliability of the food delivery application.

11. Cybersecurity Authentication System

A cybersecurity authentication system is used to verify the identity of users before allowing them to access a website or application. If the authentication system contains defects, unauthorized users may gain access to confidential information. To identify these defects, a combination of Error Guessing and Path Analysis can be used. Error Guessing is a testing technique in which testers predict possible errors based on their experience and knowledge of common mistakes. For example, testers can enter incorrect passwords repeatedly, submit empty username and password fields, use invalid usernames, and attempt to log in after an account has been locked. They can also check whether the system properly handles special characters and repeated login attempts.

Path Analysis is a white-box testing technique that examines the different execution paths inside the program. It helps testers understand how the system processes valid login details, invalid passwords, account lockouts, and password recovery requests. By testing these paths, testers can identify conditions that may allow unauthorized access or prevent legitimate users from logging in. Combining Error Guessing and Path Analysis helps detect security weaknesses and improve the reliability of the authentication system.

12.E-Learning Platform with Tiered Access

An e-learning platform provides different levels of access depending on the user's subscription plan and its expiration date. For example, users may have free, basic, or premium subscriptions, with each plan providing different learning materials and features. If the platform contains defects in subscription validation, users may lose access before their subscription expires or continue accessing premium content after expiration. To identify these defects, Equivalence Class Partitioning (ECP) and Boundary Value Analysis (BVA) can be used.

Equivalence Class Partitioning divides the input data into groups that are expected to behave similarly. For example, subscription dates can be divided into valid dates, expired dates, and invalid dates. Subscription types can also be divided into free, basic, and premium categories. Instead of testing every possible input, testers select representative values from each group. Boundary Value Analysis focuses on values near important limits. For example, testers check the day before the expiration date, the expiration date itself, and the day after expiration. They also test whether upgrading from a basic plan to a premium plan updates the user's access correctly. These techniques help identify errors in subscription management, expiration-date validation, and access control, ensuring that users receive the correct features according to their subscription status.

13.Smart-Home Lighting System

A smart-home lighting system automatically controls lights based on conditions such as motion detection, room brightness, and user settings. For example, the light may turn on when motion is detected in a dark room and remain off when the room is already bright. If the system contains complex Boolean expressions or switch-case statements, incorrect conditions may cause the lights to operate unexpectedly. To identify these defects, Branch Coverage and Condition Coverage can be used.

Branch Coverage ensures that every possible outcome of a decision is executed at least once. For example, if a decision determines whether a light should turn ON or OFF, testers must check both outcomes. Condition Coverage checks whether each individual condition in a Boolean expression evaluates to both true and false. For instance, if the system uses motion detection and room brightness to control lighting, testers should check different combinations of motion being detected or not detected and the room being bright or dark. Switch-case statements should also be tested with every valid case and the default case, if one exists. These techniques help identify errors in decision-making logic and ensure that the lighting system responds correctly under different conditions.

Important test cases:

- Detect motion in a dark room.
- Detect no motion in a dark room.
- Detect motion in a bright room.
- Test each Boolean condition as both true and false.
- Test every switch-case option and the default case, if present.

14.Logistics System for Shipping Weight Validation

A logistics system calculates shipping charges based on the weight of a package. The system must validate the entered weight before calculating the cost. If invalid weights are accepted, the system may generate incorrect shipping charges or cause processing errors. For example, a package cannot normally have a shipping weight of zero or a negative value. To identify these defects, Robustness Boundary Value Analysis can be used. This technique tests values around the minimum and maximum acceptable limits, including values just outside those limits.

For example, if the permitted shipping weight is from 1 kg to 50 kg, testers can check values such as 0.9 kg, 1 kg, 1.1 kg, 49.9 kg, 50 kg, and 50.1 kg. They should also test zero and negative weights, such as -1 kg, to confirm that the system rejects invalid inputs. If the system accepts only whole numbers, decimal values should also be tested to verify the input rules. Testers must check whether the system displays appropriate error messages and prevents invalid weights from being used in shipping calculations. These tests help identify boundary-related defects and ensure that the system validates package weights correctly before calculating charges.

15.Medical Imaging System with Parallel Processing

A medical imaging system processes medical images to help healthcare professionals examine and understand a patient's condition. Some systems use parallel processing to process large datasets more efficiently. However, defects may occur when loops execute under certain conditions or when multiple processing tasks run simultaneously. In this scenario, the system produces inconsistent results only when the dataset size is a prime number. This may indicate a defect in loop control, data distribution, or parallel processing logic. To investigate the problem, Basis Path Testing can be used as a white-box testing technique.

Basis Path Testing examines the internal control flow of a program using a Control Flow Graph (CFG). The graph represents the sequence of operations, decisions, loops, and possible execution paths. Testers calculate the cyclomatic complexity of the program to determine the number of independent paths that should be tested. They then design test cases to execute these paths and check whether the program behaves correctly. Special attention should be given to loop entry conditions, loop termination conditions, and the way data is divided among parallel tasks.

Testers should run the system using different dataset sizes, including prime and non-prime numbers, to investigate why the defect occurs only with prime-sized datasets. They should compare the output across repeated executions to identify inconsistencies and examine whether all data items are processed correctly. Since parallel processing can introduce timing-related problems, Basis Path Testing should be supplemented with concurrency-focused tests where necessary. This approach helps identify defects in program control flow and provides a structured way to investigate inconsistent medical imaging results**.**

Important test cases:

- Process a dataset containing 4 items.
- Process a dataset containing 5 items (prime number).
- Process a dataset containing 6 items.

- Process a dataset containing 7 items (prime number).
- Process the same dataset multiple times and compare the results.
- Verify that every item is processed exactly once.
- Test loop termination and parallel task execution.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification